

1.2 scanf 与 printf

格式符	可用在	对应类型	说明
%d	scanf / printf	int	十进制整数
%u	scanf / printf	unsigned int	无符号整数
%lld	scanf / printf	long long	必须写两个 l
%llu	scanf / printf	unsigned long long	无符号长整型
%f	printf	float / double	printf 里 double 也用 %f
%lf	scanf	double	scanf 里 double 必须用 %lf
%c	scanf / printf	char	scanf 会读入空白字符
%s	scanf / printf	char 数组	遇空白停止，自动补反斜杠零
%x	scanf / printf	int	十六进制
%o	scanf / printf	int	八进制
%%	printf	无	输出一个百分号

写法	效果
%5d	占 5 位，右对齐，不足补空格
%-5d	占 5 位，左对齐
%05d	占 5 位，补前导零，例如 007
%.3f	保留 3 位小数，会四舍五入
%8.3f	总宽 8 位，其中小数 3 位
%.10s	只输出前 10 个字符

要点

scanf("%c", &c) 会读到上一行残留的换行或空格。想读一个非空白字符，写成 scanf(" %c", &c) ，百分号前面加一个空格表示跳过所有空白字符。

scanf 的参数必须传地址，scanf("%d", &n) ；数组名本身就是地址，scanf("%s", buf) 不要加取址

符。
精确四舍五入时可以写成 `printf("%.2f", x + 1e-9)` 补救。

1.3 cin / cout 与 getline

```
int main() {
    ios::sync_with_stdio(false);
    cin.tie(0);

    int n; cin >> n;
    string s;
    cin.ignore();          // 丢掉上一行残留的换行
    getline(cin, s);       // 读入整行，含空格

    cout << fixed << setprecision(2) << 3.14159 << '\n';    // 输出 3.14
}
```

`ios::sync_with_stdio(false); cin.tie(0);` 让 cin/cout 接近 scanf/printf 的速度。这两行必须放在**所有输入输出之前**，写在 main 开头。

`getline(cin, s, ',')` 可以指定分隔符，常用于按逗号拆分行。

输出格式用 `cout << fixed << setprecision(k) << x;` 保留 k 位小数，需要头文件 `iomanip`, `bits` 已经包含。

`cout` 默认只保留 6 位**有效数字**，输出大整数或需要固定小数位时必须用 `fixed << setprecision`。

1.4 输出格式

保留小数位用 `printf("%.4f\n", ans);`。

补前导零用 `printf("%02d:%02d\n", h, m);`，输出形如 `09:05`。

整数除法是向零取整：`7 / 2 == 3`，`-7 / 2 == -3`。需要向上取整且 `a`、`b` 都为正时用 `(a + b - 1) / b`。

要输出一个百分号写 `printf("%%")`

第二章 容器与算法库函数接口

我要做的事	用什么	复杂度
连续存储、随机访问	vector	访问 O(1)，尾插平均 O(1)

我要做的事	用什么	复杂度
头尾都要增删	deque	头尾 $O(1)$
后进先出	stack	增删 $O(1)$
先进先出	queue	增删 $O(1)$
每次取最大或最小	priority_queue	入堆出堆 $O(\log n)$
自动去重且有序	set	增删查 $O(\log n)$
键值映射、要有序	map	$O(\log n)$
键值映射、不需要有序	unordered_map	平均 $O(1)$
只判存在、去重	unordered_set	平均 $O(1)$
字符串处理	string	见 2.3

2.2 vector

```
vector<int> v; // 空
vector<int> v(n); // n 个 0
vector<int> v(n, -1); // n 个 -1
vector<int> v = {1, 2, 3};
vector<vector<int>> g(n); // n 行，每行是空 vector，建图常用
vector<vector<int>> mat(n, vector<int>(m, 0)); // n 行 m 列全 0
```

接口一览

方法	作用	返回值	复杂度
v.size()	元素个数	size_t, 无符号	$O(1)$
v.empty()	是否为空	bool	$O(1)$
v.push_back(x)	尾部插入 x	void	平均 $O(1)$
v.pop_back()	删除尾部元素	void	$O(1)$
v.front() / v.back()	首元素 / 末元素	引用	$O(1)$
v[i]	下标访问	引用, 不检查越界	$O(1)$

方法	作用	返回值	复杂度
v.at(i)	下标访问	引用，越界抛异常	O(1)
v.clear()	清空，size 变 0，容量不变	void	O(n)
v.resize(n)	改成 n 个元素，多出的补 0	void	O(n)
v.resize(n, x)	多出的补 x	void	O(n)
v.reserve(n)	预分配容量，size 不变	void	O(n)
v.insert(it, x)	在迭代器 it 前插入 x	指向新元素的迭代器	O(n)
v.erase(it)	删除 it 指向的元素	指向下一个元素的迭代器	O(n)
v.erase(it1, it2)	删除左闭右开区间	指向下一个元素的迭代器	O(n)

空 vector 上调用 pop_back()、front()、back() 是未定义行为，先判 empty()。

2.3 string

```
string s; // 空串
string s = "hello";
string s("hello");
string s(5, 'a'); // "aaaaa", 5 个字符 'a'
string t(s); // 拷贝
string t(s, 1, 3); // 从 s 的下标 1 开始取 3 个字符
string t(s.begin(), s.end()); // 用迭代器区间构造
char buf[100] = "abc";
string t(buf); // 从 C 字符串构造
string t(10, 0); // 10 个 '\0', 某些题初始化用得上
```

方法	作用	返回值
s.size() 或 s.length()	字符个数，两者完全等价	size_t，无符号
s.empty()	是否为空串	bool
s[i]	第 i 个字符，下标从 0 开始	char 的引用，不检查越界
s.at(i)	第 i 个字符	char 的引用，越界抛 out_of_range
s.front() / s.back()	首字符 / 末字符	char 的引用

方法	作用	返回值
s.c_str()	转成 C 风格字符串	const char*, printf 必须用它
s.data()	同上	字符指针

```
string s = "hello";
cout << s[0];           // h
cout << s.back();        // o
s[0] = 'H';              // 可以直接改，得到 "Hello"
printf("%s\n", s.c_str()); // 输出必须用 c_str()
```

方法	作用	返回值
s += t	尾部拼接 t	string 的引用
s.append(t)	尾部拼接 t	string 的引用
s.push_back(c)	尾部追加一个字符	void
s.pop_back()	删除最后一个字符	void
s.insert(pos, t)	在下标 pos 处插入 t	string 的引用
s.erase(pos, len)	从 pos 开始删 len 个字符	string 的引用
s.erase(pos)	从 pos 删到末尾	string 的引用
s.replace(pos, len, t)	把从 pos 起 len 个字符替换成 t	string 的引用
s.clear()	清空	void
s.resize(n) / s.resize(n, c)	改长度，多出的补 c	void
s.assign(t)	整体赋值	string 的引用

注意：insert、erase、replace 的第一个参数都是**下标**，vector 的 insert 和 erase 收的是迭代器。

```
string t = s.substr(pos, len); // 从下标 pos 开始，取 len 个字符
string t = s.substr(pos);      // 从 pos 一直取到末尾
```

返回一个**新的 string**，原串不变。

方法	作用	返回值
<code>s.find(t)</code>	t 第一次出现的位置	下标，找不到返回 <code>string::npos</code>
<code>s.find(t, pos)</code>	从下标 pos 开始往后找	同上
<code>s.rfind(t)</code>	t 最后一次出现的位置	同上
<code>s.find_first_of(set)</code>	第一个属于字符集合 set 的字符位置	同上
<code>s.find_last_of(set)</code>	最后一个属于 set 的字符位置	同上
<code>s.find_first_not_of(set)</code>	第一个不属于 set 的字符位置	同上
<code>s.find_last_not_of(set)</code>	最后一个不属于 set 的字符位置	同上

`string::npos` 是一个静态常量，值大约是 18446744073709551615。判断有没有找到必须这样写：

```
if (s.find("abc") != string::npos) { /* 找到了 */ }
size_t pos = s.find("abc");
if (pos == string::npos) cout << "没找到";
```

`find_first_of` 的参数是"字符集合"，不是子串。例如 `s.find_first_of(",; ")` 找的是第一个逗号、分号或空格，而不是找这三个字符组成的子串。

```
if (s1 == s2) { }
if (s1 < s2) { }
int r = s1.compare(s2); // 小于返回负数，等于返回 0，大于返回正数
```

```
int i = stoi("123"); // 123
long long l = stoll("123456789012"); // 超过 int 范围必须用 stoll
double d = stod("3.14");
float f = stof("3.14");
string s = to_string(123); // "123"
string t = to_string(3.14); // "3.140000"，注意是 6 位小数
int base2 = stoi("1011", nullptr, 2); // 按二进制解析 → 11
size_t used = 0;
int v = stoi("123abc", &used); // v = 123, used = 3
```

```

int d = c - '0';           // 字符 '7' 转数字 7, 只对 '0'~'9' 有效
char c = '0' + d;         // 数字 7 转字符 '7'
int x = c - 'a';           // 'c' 转 2, 字母映射到 0~25

```

```

for (int i = 0; i < (int)s.size(); i++) cout << s[i];
reverse(s.begin(), s.end());           // 反转整个串
bool pal = (s == string(s.rbegin(), s.rend())); // 判回文

```

```

cin >> s;                          // 遇空白字符停止
getline(cin, s);                    // 读整行, 包含空格
getline(cin, s, ';');               // 读到分号为止
cout << s << '\n';
printf("%s\n", s.c_str());          // printf 必须配 c_str()

```

```

// 删除所有空格
s.erase(remove(s.begin(), s.end(), ' '), s.end());
// 统计每个字符出现次数
int cnt[256] = {0};
for (unsigned char c : s) cnt[c]++;
//进制转换
int d = x % b;
r += (d < 10 ? char('0' + d) : char('A' + d - 10));
x /= b;

```

2.4 deque

```

deque<int> dq;
dq.push_front(x); // 头部插入, 要判空
dq.pop_front();   // 删除头部

```

2.5 pair

```
pair<int,int> p = {1, 2};
pair<int,string> q = make_pair(3, "abc");
pair<int,int> r(4, 5);

p.first;           // 1
p.second;          // 2
p = {5, 6};        // 整体赋值
swap(p, r);         // 交换两个 pair

vector<pii> v = {{3,1}, {1,5}, {1,2}};
sort(v.begin(), v.end());           // (1,2) (1,5) (3,1)
sort(v.begin(), v.end(), greater<pii>()); // 反序
```

2.6 stack 与 queue

```
stack<int> st;
st.push(x);    // 入栈, 返回 void
st.pop();      // 弹出栈顶, 返回 void, 不返回被弹出的元素
st.top();      // 取栈顶, 返回引用
st.size();     // 元素个数
st.empty();    // 是否为空

queue<int> q;
q.push(x);     // 入队, 返回 void
q.pop();       // 出队, 返回 void, 不返回被弹出的元素
q.front();     // 取队首, 返回引用
q.back();      // 取队尾, 返回引用
q.size(); q.empty();
```

front()、back()、top() 返回的是引用，int x = q.front(); 这种写法取到的是拷贝。

2.7 priority_queue

```
priority_queue<int> pq;           // 大根堆, top() 是最大值
priority_queue<int, vector<int>, greater<int>> q; // 小根堆, top() 是最小值
priority_queue<pii, vector<pii>, greater<pii>> q2; // pair 小根堆, 先比 first
```

```

pq.push(x);          // 入堆,  $O(\log n)$ , 返回 void
pq.pop();            // 弹出堆顶,  $O(\log n)$ , 返回 void, 不返回元素
pq.top();            // 取堆顶, 返回常量引用
pq.size(); pq.empty();

```

2.8 set

```

set<int> s;           // 自动去重 + 升序
multiset<int> ms;     // 升序, 允许重复

s.insert(x);          // 插入, 返回 pair<迭代器, bool>, bool 表示是否真的插进去了
s.erase(x);           // 删除所有等于 x 的元素, 返回被删除的个数
s.erase(it);          // 删除迭代器 it 指向的那一个元素, 返回下一个位置的迭代器
s.count(x);            // 返回 0 或 1 (multiset 返回出现次数)
s.find(x);             // 返回指向 x 的迭代器, 找不到返回 s.end()
s.lower_bound(x);      // 第一个大于等于 x 的位置, 返回迭代器
s.upper_bound(x);      // 第一个大于 x 的位置, 返回迭代器
s.size(); s.empty(); s.clear();
*s.begin();           // 最小值
*s.rbegin();          // 最大值

```

`s.erase(it)` 里 if 判断不能省, 直接删 `end()` 是未定义行为。

```

set<int> s = {3, 1, 2};
for (int x : s) cout << x << ' ';          // 输出 1 2 3, 自动有序

auto it = s.find(2);
if (it != s.end()) cout << *it;            // 找到才能解引用

```

2.9 map

```

map<string, int> mp;      // 键升序排列, 键唯一

```

```

mp["abc"] = 1;           // 不存在就创建（值初始化为 0），存在就覆盖
mp.insert({"abc", 1});  // 已存在时不覆盖，返回 pair<迭代器, bool>
mp.count("abc");        // 返回 0 或 1 (multimap 返回个数)
mp.find("abc");         // 返回迭代器，找不到返回 mp.end()
mp.erase("abc");       // 按键删除
mp.erase(it);          // 按迭代器删除一个
mp.size(); mp.empty(); mp.clear();

if (mp.count("abc")) { ... }           // 写法一
auto it = mp.find("abc");              // 写法二
if (it != mp.end()) { int v = it->second; }

for (auto &p : mp) cout << p.first << " " << p.second << '\n';    // 键升序
for (auto it = mp.begin(); it != mp.end(); ++it) cout << it->first << " " << it->second;

```

2.10 unordered_map 与 unordered_set

```

unordered_map<string,int> ump;
unordered_set<int> us;

```

接口和 map / set 几乎一样，区别只有三点：

2.12 算法库函数接口

所有接收两个迭代器的函数，区间都是**左闭右开** [begin, end)。

```

sort(v.begin(), v.end(), greater<int>());           // 降序
sort(a, a + n);                                     // 数组版本，a 是数组名
sort(v.begin(), v.end(), [](int x, int y) { return x > y; }); // 自定义，TRUE时x在y前面

```

参数是起点、终点、可选比较器。返回值是 void。复杂度 $O(n \log n)$

二分查找

```

auto it = lower_bound(v.begin(), v.end(), x); // 第一个 >= x 的位置
auto it = upper_bound(v.begin(), v.end(), x); // 第一个 > x 的位置
bool ok = binary_search(v.begin(), v.end(), x); // x 是否存在

```

返回值都是迭代器。数组版本返回指针，要减数组名才得到下标：

查找与计数

```
find(v.begin(), v.end(), x);           // 返回第一个等于 x 的迭代器，找不到返回 end()
count(v.begin(), v.end(), x);          // 返回等于 x 的元素个数
count_if(v.begin(), v.end(), pred);    // 返回让 pred 为真的元素个数
```

这些都是 $O(n)$ 的线性扫描。要在有序容器里查找用 `set / map` 自带的 `find`，那是 $O(\log n)$ 。

修改类

```
reverse(v.begin(), v.end());           // 原地反转
rotate(v.begin(), v.begin() + k, v.end()); // 循环左移 k 位
fill(v.begin(), v.end(), x);           // 全部赋值为 x
copy(src.begin(), src.end(), dst.begin()); // 拷贝，dst 必须有足够空间
remove(v.begin(), v.end(), x);          // 把等于 x 的元素移到后面，返回新的逻辑结尾
```

```
v.erase(remove(v.begin(), v.end(), x), v.end()); // 删除所有等于 x 的元素
v.erase(remove_if(v.begin(), v.end(), pred), v.end()); // 按条件删除
```

去重一行写法（已排序）：`v.erase(unique(v.begin(), v.end()), v.end());`；

```
max(a, b);           // 返回较大值，要求两个参数类型相同
min(a, b);
max_element(v.begin(), v.end()); // 返回指向最大元素的迭代器
min_element(v.begin(), v.end()); // 返回指向最小元素的迭代器
*max_element(v.begin(), v.end()); // 解引用得到值
```

数值

```
__gcd(a, b);           // 老写法的最大公约数
gcd(a, b);             // C++17，返回最大公约数，要求非负
lcm(a, b);             // C++17，返回最小公倍数
abs(x);               // 整数绝对值，注意对 INT_MIN 无效
llabs(x);             // long long 绝对值
```

数学函数

```
sqrt(x);pow(x, y);fabs(x);
ceil(x);      // 向上取整, 返回 double
floor(x);     // 向下取整, 返回 double
round(x);     // 四舍五入, 返回 double
log(x); log2(x); log10(x); exp(x);
sin(x); cos(x); tan(x);
```

这些函数都返回 double。 `int r = (int)(sqrt(x) + 1e-9);`。

字符判断

```
isdigit(c);    // 是否是数字字符 '0'~'9', 返回非零表示真
isalpha(c);    // 是否是字母
isalnum(c);    // 是否是字母或数字
isspace(c);    // 是否是空白字符（空格、换行、制表符）
isupper(c);    // 是否是大写字母
islower(c);    // 是否是小写字母
toupper(c);    // 转大写, 返回 int, 要赋给 char
tolower(c);    // 转小写
```

这些函数的返回值是**非零表示真**，不是严格的 1。参数传 char 时要先转成 unsigned char，否则遇到负值（中文、扩展 ASCII）是未定义行为，安全写法是 `isdigit((unsigned char)c)`。

第三章 基础算法与常用技巧

3.1 排序与自定义比较器

适用情形：需要按多个关键字排序、需要自定义顺序、排序后贪心、排序后去重、排序后二分。比较器回答"第一个参数是否应该排在第二个参数前面"。

```
struct Node { int a, b; };
// 写法三: lambda, 写在调用处, 最灵活
sort(v.begin(), v.end(), [](const Node &x, const Node &y) {
    if (x.a != y.a) return x.a < y.a;
    return x.b > y.b;
});
```

3.2 二分

适用情形：题目说"最大值最小"、"最小值最大"、"至少需要多少"、"最多能装多少"；数据已经有序或者可以去重排序；

答案具有单调性，即"如果 x 可行，那么比 x 更宽松的值也可行"。

核心思想：把答案可能取值的范围分成两半，每次丢掉确定不可能的那一半。或者把"求最优解"转化成"判断某个值是否可行"，然后二分这个值。

复杂度：二分本身 $O(\log n)$ ，每次判定如果 $O(n)$ ，总共 $O(n \log n)$ 。

整数二分（求最小的满足条件的位置）

```
int l = 1, r = n;
while (l < r) {
    int mid = (l + r) / 2;          // 下取整
    if (check(mid)) r = mid;        // mid 可行，答案在左边或就是 mid
    else l = mid + 1;              // mid 不可行，答案在右边
}
// 循环结束时 l == r，就是第一个满足 check 的位置
```

整数二分（求最大的满足条件的位置）

```
int l = 1, r = n;
while (l < r) {
    int mid = (l + r + 1) / 2;      // 上取整，必须 +1
    if (check(mid)) l = mid;        // mid 可行，答案在右边或就是 mid
    else r = mid - 1;              // mid 不可行，答案在左边
}
// 循环结束时 l == r，就是最后一个满足 check 的位置
```

两套模板的区别与记忆口诀

写 $l = mid$ 的分支时， mid 必须写成 $(l + r + 1) / 2$ ，也就是**上取整**。写 $r = mid$ 的分支时， mid 写成 $(l + r) / 2$ ，下取整。

口诀：谁等于 mid ， mid 就往谁偏。

如果配错了会死循环。举例： $l = 0, r = 1, mid = (0+1)/2 = 0$ ，若分支是 $l = mid$ ，则 l 永远停在 0，循环不出来。

浮点二分

```
double lo = 0, hi = 1e9;
for (int i = 0; i < 100; i++) {    // 迭代 100 次，精度足够
    double mid = (lo + hi) / 2;
    if (check(mid)) lo = mid;      // 可行，往大找
    else hi = mid;
}
// 答案约等于 lo
```

浮点二分不要用 `while (hi - lo > eps)`，容易出现死循环或者精度不够。固定迭代 100 次是最稳的写法。

二分答案的判定函数

二分答案的关键不是二分本身，而是把“求最优解”变成“判断某个值是否可行”。

`check(mid)` 的边界要能覆盖答案本身。例如答案是 0 也可能成立，那么 1 的初值就应该是 0 而不是 1。

3.3 前缀和与差分

适用情形：多次询问区间和、区间最值（前缀和的变体）、区间加同一个数、统计区间被覆盖次数。看到“m 次询问”“区间修改”就该想到它。

核心思想：前缀和把“区间求和”从 $O(n)$ 降到 $O(1)$ ；差分把“区间加值”从 $O(n)$ 降到 $O(1)$ 。两者互为逆运算。

复杂度：预处理 $O(n)$ ，每次查询或修改 $O(1)$ 。

一维前缀和

```
// a[1..n], pre[0] = 0
for (int i = 1; i <= n; i++) pre[i] = pre[i - 1] + a[i];
// 区间 [l, r] 的和
ll sum = pre[r] - pre[l - 1];
```

二维前缀和

```
// a[1..n][1..m]
for (int i = 1; i <= n; i++)
    for (int j = 1; j <= m; j++)
        s[i][j] = s[i-1][j] + s[i][j-1] - s[i-1][j-1] + a[i][j];

// 子矩阵 (x1,y1) 到 (x2,y2) 的和
ll sum = s[x2][y2] - s[x1-1][y2] - s[x2][y1-1] + s[x1-1][y1-1];
```

一维差分

用途：对区间 $[1, r]$ 的每个元素加 v ，重复 m 次，最后问每个元素的值。

```
// 差分数组 d，初始全 0（差分对应原数组 a 全 0）
d[1] += v;
d[r + 1] -= v;

// 最后还原
for (int i = 1; i <= n; i++) {
    d[i] += d[i - 1];
    a[i] = d[i];          // 或直接用 d[i] 当结果
}

for (int i = 1; i <= n; i++) d[i] = a[i] - a[i-1];
```

二维差分

用途：对子矩阵 $(x1,y1)$ 到 $(x2,y2)$ 整体加 v 。

```
d[x1][y1] += v;
d[x1][y2+1] -= v;
d[x2+1][y1] -= v;
d[x2+1][y2+1] += v;
// 还原：对 d 做一次二维前缀和即可
for (int i = 1; i <= n; i++)
    for (int j = 1; j <= m; j++)
        d[i][j] += d[i-1][j] + d[i][j-1] - d[i-1][j-1];
```

3.4 双指针与滑动窗口

适用情形：在序列里找"最长的满足某条件的连续子段"、"最短的覆盖某集合的连续子段"、"元素和不超过 k 的最长子段"。条件是**单调**的：右端点往右移会让条件更难满足，左端点往右移会让条件更容易满足。

核心思想：左右两个指针都只往右走，不回头，所以总移动次数 $O(n)$ 。右指针负责扩张窗口，左指针负责在窗口不合法时收缩。

复杂度： $O(n)$ 。

滑动窗口模板

```
int l = 0, ans = 0;
for (int r = 0; r < n; r++) {
    // 一、把 a[r] 加入窗口，更新窗口的统计数据
    add(a[r]);
    // 二、只要窗口不合法，就收缩左边界
    while (/* 窗口不合法 */) {
        remove(a[l]);
        l++;
    }
    // 三、此时窗口合法，更新答案
    ans = max(ans, r - l + 1);
}
```

实例：和不超过 k 的最长连续子段（所有元素为正数）

```
ll sum = 0; int l = 0, ans = 0;
for (int r = 0; r < n; r++) {
    sum += a[r];
    while (sum > k) { sum -= a[l]; l++; }
    ans = max(ans, r - l + 1);
}
```

这里能这么写的前提是元素都是正数。有负数时窗口和不单调，必须换方法（前缀和 + 二分或者单调队列）。

对撞双指针

适用于**有序数组**求两数之和、三数之和、去重后的配对。

```

int l = 0, r = n - 1;
while (l < r) {
    int s = a[l] + a[r];
    if (s == target) { /* 找到 */ l++; r--; }
    else if (s < target) l++;      // 和太小，左指针右移
    else r--;                    // 和太大，右指针左移
}

```

3.5 离散化

适用情形：值的范围很大（例如到 $1e9$ ），但实际出现的数只有 n 个（ n 到 $1e5$ ）。需要按值做下标、用树状数组、开数组统计、二分查找。

核心思想：把"值本身"换成"它在所有出现过的值里排第几"，排名是 1 到 n 的连续整数，可以当下标用。

复杂度：排序 $O(n \log n)$ ，每次查询 $O(\log n)$ 。

三步模板

```

vector<int> val(a, a + n);          // 把要离散化的值拷进来
sort(val.begin(), val.end());
val.erase(unique(val.begin(), val.end()), val.end());

// 把 x 映射成 1 到 m 的排名
int id = lower_bound(val.begin(), val.end(), x) - val.begin() + 1;

// 反过来，把排名还原成原值
int original = val[id - 1];

```

`val.size()` 就是不同值的个数 m 。用 `+1` 是为了让排名从 1 开始，方便配树状数组（树状数组必须从 1 开始）。

批量离散化

```
vector<int> raw; // 所有出现过的值
for (int i = 0; i < n; i++) raw.push_back(a[i]);
sort(raw.begin(), raw.end());
raw.erase(unique(raw.begin(), raw.end()), raw.end());

vector<int> b(n);
for (int i = 0; i < n; i++)
    b[i] = lower_bound(raw.begin(), raw.end(), a[i]) - raw.begin() + 1;
```

100, 50, 100, 200==50 100 200==2 1 2 3;统计每个数出现的次数可用

3.6 贪心常见模型

适用情形：题目要求"最少 / 最多 / 最大收益"，每一步的选择看起来局部最优就能导致全局最优，且没有明显需要"反悔"的结构。

核心思想：每一步都做当前看起来最好的选择，并且能证明后面的选择不会因此变差。

复杂度：排序是主要开销，一般 $O(n \log n)$ 。

模型一：最多能选多少个不重叠区间

排序规则：**按右端点升序**。依次扫描，能选就选。

```
sort(seg, seg + n, [](const P &x, const P &y) { return x.r < y.r; });
int cnt = 0, last = -INF;
for (int i = 0; i < n; i++)
    if (seg[i].l >= last) { cnt++; last = seg[i].r; }
```

理由：右端点越小，留给后面的空间越多。按右端点排序能保证每一步选的区域都不会让后续变差。

模型二：区间选点，用最少的点覆盖所有区间

排序规则同样按右端点升序，每次在当前区间的右端点放一个点。

模型三：区间覆盖，用最少数区间覆盖 $[1, m]$

排序规则：**按左端点升序**。每次在"左端点能接上当前覆盖位置"的所有区间里，选右端点最远的那个。

```

sort(seg, seg + n, [](const P &x, const P &y) { return x.l < y.l; });
int cur = 1, i = 0, cnt = 0;
while (cur <= m) {
    int best = cur - 1;
    while (i < n && seg[i].l <= cur) { best = max(best, seg[i].r); i++; }
    if (best < cur) { /* 覆盖不了，无解 */ break; }
    cnt++; cur = best + 1;
}

```

模型四：排序后依次选择

把元素按某个关键字排序，然后从左到右做决策。例如"每次选最大的"、"按性价比排序"、"按结束时间排序"。

模型五：Huffman 思想

每次合并当前最小的两个元素。用优先队列实现，复杂度 $O(n \log n)$ 。典型题目是合并果子、合并石子。

```

priority_queue<ll, vector<ll>, greater<ll>> pq;
ll cost = 0;
while (pq.size() > 1) {
    ll x = pq.top(); pq.pop();
    ll y = pq.top(); pq.pop();
    cost += x + y;
    pq.push(x + y);
}

```

3.7 折半枚举

适用情形： n 到 40，直接枚举 2^n 太大 ($1e12$)，但 $2^{(n/2)}$ 只有 100 万。

做法：把元素分成两半，各自枚举所有子集的结果存下来，排序后对第一半的每个结果，在第二半里二分查找能和它配成目标的个数。

3.8 位运算技巧

适用情形：状态压缩、集合运算、快速判断奇偶、取某个二进制位、统计二进制中 1 的个数。

基本操作

```
x & 1           // 取最低位，等于 1 表示奇数
x >> k & 1      // 取第 k 位（第 k 位表示 2 的 k 次方）
x | (1 << k)    // 把第 k 位置 1
x & ~(1 << k)   // 把第 k 位置 0
x ^ (1 << k)    // 把第 k 位取反
x & -x          // 取出最低位的 1（lowbit），树状数组用
```

第四章 数据结构

4.1 并查集

适用情形：题目里出现“朋友的朋友是朋友”、“两个元素是否属于同一组”、“把两堆合并起来”、“判断加这条边会不会成环”。图论里的 Kruskal 算法也要用。

核心思想：每个集合用一棵树表示，树根是这个集合的代表。判断两个元素是否同组就看它们的根是否相同。合并两个集合就是把一棵树的根挂到另一棵树的根上。

```
const int MAXN = 200005;
int fa[MAXN];

void init(int n) { // 下标从 1 开始，初始化 n 个独立集合
    for (int i = 1; i <= n; i++) fa[i] = i;
}
int find(int x) { // 返回 x 所在集合的根
    if (fa[x] == x) return x;
    return fa[x] = find(fa[x]); // 递归过程中顺手把路径上的点直接挂到根上
}
void unite(int a, int b) { // 合并 a 和 b 所在的两个集合
    fa[find(a)] = find(b);
}
bool isSame(int a, int b) { // 判断是否在同一集合
    return find(a) == find(b);
}
```

按秩合并版本

路径压缩已经足够快。如果需要在极深的链上避免递归爆栈，或者需要维护集合大小，用按大小合并：

```

int sz[MAXN];

void init(int n) {
    for (int i = 1; i <= n; i++) { fa[i] = i; sz[i] = 1; }
}

void unite(int a, int b) {
    a = find(a); b = find(b);
    if (a == b) return;
    if (sz[a] < sz[b]) swap(a, b);    // 让 a 成为较大的那棵树
    fa[b] = a;
    sz[a] += sz[b];
}

```

4.2 树状数组

适用情形：需要单点修改 + 区间求和，或者区间修改 + 单点查询，或者区间修改 + 区间求和。数据量到 $1e5$ 时，每次都暴力 $O(n)$ 会超时。如果只需要查询不需要修改，用前缀和就够了。

核心思想：用 `lowbit` 把数组划分成若干个区间，每个 `c[i]` 负责长度是 `lowbit(i)` 的一段。修改时沿着 `i + lowbit(i)` 往上更新所有覆盖到 `i` 的位置，查询时沿着 `i - lowbit(i)` 往下累加。

复杂度：建树 $O(n \log n)$ ，单次修改和单次查询都是 $O(\log n)$ 。

单点修改 + 区间求和

```
const int MAXN = 200005;
ll c[MAXN];
int n;
int lowbit(int x) { return x & -x; }
void add(int i, ll v) { // 在下标 i 处加上 v
    for (; i <= n; i += lowbit(i)) c[i] += v;
}
ll sum(int i) { // 求 a[1] + ... + a[i]
    ll s = 0;
    for (; i > 0; i -= lowbit(i)) s += c[i];
    return s;
}
ll rangeSum(int l, int r) { return sum(r) - sum(l - 1); }

int main(){
    cin >> n;
    vector<ll> a(n + 1);
    for (int i = 1; i <= n; ++i) {
        cin >> a[i];
        add(i, a[i]); // 建树：逐个加入
    }
}
```

接口说明

`add(i, v)` 把位置 `i` 的值增加 `v`，返回 `void`。初始值通过调用 `n` 次 `add(i, a[i])` 建立起来。

`sum(i)` 返回前缀和 `a[1] + a[2] + ... + a[i]`。

`rangeSum(l, r)` 返回区间和，内部就是两个前缀和相减。

下标必须从 1 开始。这是树状数组最大的限制。如果原题下标从 0 开始，读进来时统一加 1。

区间修改 + 单点查询

用差分的思想，树状数组维护的是差分数组,其余代码可复用，把a换成b即可：

```
// 给区间 [l, r] 每个数加上 v
add(l, v);
add(r + 1, -v);
// 查询位置 i 的值
ll value = sum(i);
```

4.4 单调栈与单调队列

适用情形

单调栈：找每个元素**左右两边第一个比它大或比它小的元素**、求直方图中最大矩形面积、柱状图接雨水。

单调队列：求**滑动窗口的最大值或最小值**、优化 DP 转移、求"区间长度不超过 k 的最大子段和"。

核心思想

两者都维护一个内部的单调结构。新元素进来时，把"已经没用的元素"从尾部弹掉，保持结构单调。单调栈里存下标，栈内下标递增、对应的值单调。单调队列用 deque 实现，队首是当前窗口的最值，队尾是最后加入的元素。

复杂度：每个元素最多入栈（队）一次、出栈（队）一次，总复杂度 $O(n)$ 。

单调栈：求每个元素右边第一个更大的元素

```
stack<int> st;                // 存下标
int nxt[MAXN];               // nxt[i] 是 i 右边第一个比 a[i] 大的位置
memset(nxt, 0, sizeof(nxt)); // 0 表示不存在

for (int i = 1; i <= n; i++) {
    while (!st.empty() && a[st.top()] < a[i]) {
        nxt[st.top()] = i;        // i 就是 st.top() 右边第一个更大的
        st.pop();
    }
    st.push(i);
}
```

思路：栈里维护一个"还没找到右边更大元素"的下标序列，它们的值从栈底到栈顶是递减的。新来的 $a[i]$ 如果比栈顶大，说明栈顶元素的答案就是 i ，可以弹出来记录答案；一直弹到栈顶不小于 $a[i]$ 为止，再把 i 压进去。

想求"右边第一个更小的"，把比较改成 $a[st.top()] > a[i]$ 。想求"左边第一个更大的"，把数组反过来做一遍，或者从右往左扫描。

单调栈：直方图最大矩形

```
// h[1..n] 是每个柱子的高度，求能画出的最大矩形面积
stack<int> st; st.push(0);          // 0 号位置高度视为 0，作为哨兵
h[0] = 0;
ll ans = 0;
for (int i = 1; i <= n + 1; i++) {
    int cur = (i == n + 1) ? 0 : h[i];    // 末尾补一个 0 高度，把栈清空
    while (!st.empty() && h[st.top()] > cur) {
        int height = h[st.top()]; st.pop();
        int left = st.top();              // 左边第一个比 height 矮的位置
        ans = max(ans, (ll)height * (i - left - 1));
    }
    st.push(i);
}
```

单调队列：滑动窗口最大值

```
deque<int> q;                      // 存下标，对应的值从队首到队尾递减
vector<int> res(n - k + 2);

for (int i = 1; i <= n; i++) {
    // 一、队首如果已经滑出窗口，弹出
    while (!q.empty() && q.front() <= i - k) q.pop_front();
    // 二、新元素入队前，把队尾所有不大于它的元素弹掉
    while (!q.empty() && a[q.back()] <= a[i]) q.pop_back();
    // 三、新元素入队
    q.push_back(i);
    // 四、窗口长度够 k 了，记录答案
    if (i >= k) res[i - k + 1] = a[q.front()];
}
```

接口说明： k 是窗口长度， $res[1..n-k+1]$ 里存每个窗口的最大值。求最小值时把第二步的比较改成 $a[q.back()] \geq a[i]$ 。

为什么每个元素只进出一次： 第二步的 while 循环一旦弹出元素就再也不会放回来，所以整个 for 循环里 `pop_back` 的总次数不超过 n ，均摊下来是 $O(n)$ 。

单调队列优化 DP

适用情形：转移方程形如 $dp[i] = \min(dp[j] + something(i))$ ，其中 j 被限制在一个**滑动窗口**里（例如

$i - k \leq j \leq i - 1$), 而且 $dp[j]$ 的候选值随 j 有单调性。

做法：把上面模板里的 $a[]$ 换成 $dp[]$ ，队首就是窗口内最优的转移来源。

判断"是否出窗口"必须用 `while` 而不是 `if`，而且这个判断要在入队之前做，保证队首始终是有窗口内的元素。

单调队列里存的是下标，因为要靠下标判断是否滑出窗口，直接存值的话判断不了。

单调栈和单调队列都只在"答案只和局部有关"时能用。如果题目问的是区间的整体性质（比如区间和），用前缀和或线段树。

第五章 图论

5.1 存图三种方式

适用情形：只要题目里出现节点和边，就要先决定怎么存。

方式一：邻接矩阵

```
int g[1005][1005];           // n 不超过 1000 时可用
// 初始化全 0，无边
g[u][v] = 1;                  // 有向边 u -> v
g[u][v] = g[v][u] = 1;        // 无向边
// 带权图：g[u][v] = w，无边时初始化成 INF
// 判断边是否存在：g[u][v] != 0
```

优点： $O(1)$ 判断任意两点之间有没有边，写 Floyd 最方便。

缺点：空间是 $O(n^2)$ ， n 到 $1e5$ 时开不下。带权图无边时要初始化成 INF，不能是 0。

方式二：vector 邻接表（推荐）

```

vector<int> g[MAXN]; // 无权图: g[u] 存 u 的所有邻居
vector<pair<int,int>> gw[MAXN]; // 带权图: g[u] 存 (邻接点, 边权)
// 加一条有向边 u -> v
g[u].push_back(v);
gw[u].push_back({v, w});
// 加一条无向边
g[u].push_back(v); g[v].push_back(u);
gw[u].push_back({v, w}); gw[v].push_back({u, w});
// 遍历 u 的所有出边
for (int v : g[u]) { /* 无权图 */ }
for (auto &e : gw[u]) {
    int v = e.first, w = e.second;
}

```

5.2 DFS 与 BFS

适用情形

DFS：连通块计数、判断两点是否连通、求环、暴力搜索、拓扑排序（另一种写法）、树上的各种统计。

BFS：**无权图的最短步数**、多源最短、网格上的最短路、层序遍历。只要边权都相等，求最短路就用 BFS。

核心思想：DFS 一条路走到黑再回溯；BFS 一圈一圈往外扩，所以第一次到达某个点时走过的步数就是最短的。

DFS 求连通块

```

int vis[MAXN];
void dfs(int u) {
    vis[u] = 1;
    for (int v : g[u]) {
        if (!vis[v]) dfs(v);
    }
}
// 统计连通块个数
int comp = 0;
for (int i = 1; i <= n; i++)
    if (!vis[i]) { comp++; dfs(i); }

```

接口说明：vis 数组标记是否访问过，初始全 0。dfs(u) 从 u 出发把所有能到的点标记为已访问。连通块个数就是外层循环里调用 dfs 的次数。

BFS 求无权图最短路

```

int dist[MAXN];
void bfs(int s, int n) {
    memset(dist, -1, sizeof(dist));    // -1 表示未访问
    queue<int> q;
    dist[s] = 0;
    q.push(s);
    while (!q.empty()) {
        int u = q.front(); q.pop();
        for (int v : g[u]) {
            if (dist[v] == -1) {        // 第一次到达就是最短
                dist[v] = dist[u] + 1;
                q.push(v);
            }
        }
    }
}

```

接口说明： dist[i] 表示从起点 s 到 i 的最短步数，不可达时保持 -1。调用方式 bfs(1, n); ，之后直接用 dist[目标]。

入队时就把 dist 设好（而不是出队时），这样才能保证同一个点只被入队一次。

网格图上的 BFS

```

int dx[4] = {-1, 1, 0, 0};
int dy[4] = {0, 0, -1, 1};
// 从 (sx, sy) 出发, '.' 可走, '#' 是墙
int dist[1005][1005];
memset(dist, -1, sizeof(dist));
queue<pair<int,int>> q;
dist[sx][sy] = 0;
q.push({sx, sy});
while (!q.empty()) {
    auto [x, y] = q.front(); q.pop();
    for (int d = 0; d < 4; d++) {
        int nx = x + dx[d], ny = y + dy[d];
        if (nx < 0 || nx >= n || ny < 0 || ny >= m) continue;
        if (map_[nx][ny] == '#') continue;
        if (dist[nx][ny] != -1) continue;
        dist[nx][ny] = dist[x][y] + 1;
        q.push({nx, ny});
    }
}

```

多源 BFS

适用情形：有多个起点，要求"距离最近的起点"的距离

做法：初始化时把所有起点都 push 进队列并把 dist 设为 0，后面和普通 BFS 完全一样。这样队列里天然按距离分层。

DFS 的递归深度：n 到 1e5 时用 BFS 或者手写栈。

5.3 最短路

看到"边权非负、求单源最短路"就用 Dijkstra；看到"求任意两点之间的距离"且 n 不大就用 Floyd；

Dijkstra

核心思想：每次从还没确定的点里挑出当前距离最小的那个，它的距离就已经是最优的了，然后拿它去更新邻居。用优先队列来快速取出最小距离。

```
typedef pair<ll,int> pli;           // (距离, 节点编号), first 是距离
vector<pair<int,ll>> g[MAXN];      // (邻接点, 边权)
ll dist[MAXN];

void dijkstra(int s, int n) {
    memset(dist, 0x3f, sizeof(dist)); // 全部设成 LINF 量级
    dist[s] = 0;
    priority_queue<pli, vector<pli>, greater<pli>> pq; // 小根堆
    pq.push({0, s});
    while (!pq.empty()) {
        auto [d, u] = pq.top(); pq.pop();
        if (d > dist[u]) continue; // 过期条目, 跳过
        for (auto &e : g[u]) {
            int v = e.first; ll w = e.second;
            if (dist[u] + w < dist[v]) {
                dist[v] = dist[u] + w;
                pq.push({dist[v], v});
            }
        }
    }
}
```

接口说明：dijkstra(s, n) 计算从 s 出发到所有点的最短距离，结果存在全局数组 dist 里。不可达的点 dist 保持初始的"无穷大"，判断时可以用 if (dist[v] >= LINF / 2)。

优先队列里存的是 pair<ll,int>，靠 greater<pli> 使 first 最小的先出堆。如果写成默认的 priority_queue<pli>（大根堆），取出来的是距离最大的点，算法就错了。

if (d > dist[u]) continue; 这一行不能省。同一个点可能被多次入堆，堆里的旧记录对应的是更大的

距离，直接跳过。

复杂度 $O((n + m) \log n)$ 。n 到 $1e5$ 、m 到 $2e5$ 时完全够用。

Floyd

核心思想：动态规划。 $d[i][j]$ 表示从 i 到 j 的最短路，枚举中间点 k ，看能不能通过 k 让路径变短。

```
11 d[505][505];
void floyd(int n) {
    for (int k = 1; k <= n; k++)
        for (int i = 1; i <= n; i++)
            for (int j = 1; j <= n; j++)
                if (d[i][k] + d[k][j] < d[i][j])
                    d[i][j] = d[i][k] + d[k][j];
}
```

初始化

```
for (int i = 1; i <= n; i++)
    for (int j = 1; j <= n; j++)
        d[i][j] = (i == j) ? 0 : LINF;
// 然后读边: d[u][v] = min(d[u][v], w);
```

接口说明：三层循环的顺序**必须把 k 放在最外层**，这是算法正确性的关键，写反了结果就错。复杂度 $O(n^3)$ ，n 超过 500 就会超时。

自环初始化为 0、无边为 LINF，这样"从 i 到 i 的距离是 0"这条基础事实才成立。

易错点

Dijkstra 的 `dist` 数组和 `g` 数组在多组数据时要清空。 `g` 是 `vector` 数组时，用

```
for (int i = 1; i <= n; i++) g[i].clear();
```

边权用 `long long`，因为路径长度是很多条边的和，容易超 `int`。

5.4 最小生成树

适用情形：给出 n 个点和 m 条带权边，要求选 $n-1$ 条边把所有点连起来，且总权值最小。典型题面是"修路使所有村庄连通，总花费最小"。

两种算法的选择：稀疏图 (m 和 n 同量级) 用 Kruskal，稠密图 (m 接近 n^2) 用 Prim。CSP 里 m 通常不大，**优先写 Kruskal**，因为代码短且和并查集复用。

Kruskal

核心思想：把所有边按权值从小到大排序，依次考虑每条边，如果这条边的两个端点还不在于同一个连通块里，就选它并合并。选够 $n-1$ 条边就结束。用并查集判断连通性。

复杂度：排序 $O(m \log m)$ ，并查集接近 $O(1)$ 。

```
struct Edge {
    int u, v;
    ll w;
};
Edge e[MAXM];
bool cmp(const Edge &a, const Edge &b) { return a.w < b.w; }
int fa[MAXN];
int find(int x) { return fa[x] == x ? x : fa[x] = find(fa[x]); }
// 返回最小生成树的总权值；如果图不连通返回 -1
ll kruskal(int n, int m) {
    for (int i = 1; i <= n; i++) fa[i] = i;
    sort(e, e + m, cmp);
    ll total = 0;
    int used = 0;
    for (int i = 0; i < m; i++) {
        int ru = find(e[i].u), rv = find(e[i].v);
        if (ru == rv) continue;          // 成环，跳过
        fa[ru] = rv;                     // 合并
        total += e[i].w;
        if (++used == n - 1) break;      // 已经选够
    }
    return (used == n - 1) ? total : -1;
}
```

接口说明：参数是点数和边数，返回最小生成树的总权值。图不连通时无法生成树，返回 -1（调用方要按题目要求输出"无法连通"之类的信息）。

边的下标用 $e[0..m-1]$ ，和 `sort` 的写法配套。

易错点

并查集必须在函数内部先初始化，不能依赖全局的旧数据。

`used` 计数器不能省，用来判断图是否连通。

边权用 `long long`， m 条边求和可能超过 `int`。

Prim（简要）

核心思想：从任意一个点开始，每次选一条"一端已经在树里、另一端还没进树"的权值最小的边，把新点加进树。用优先队列维护候选边。

```
ll prim(int n) {
    vector<int> vis(n + 1, 0);
    priority_queue<pli, vector<pli>, greater<pli>> pq;
    pq.push({0, 1});
    ll total = 0;
    int cnt = 0;
    while (!pq.empty() && cnt < n) {
        auto [w, u] = pq.top(); pq.pop();
        if (vis[u]) continue;
        vis[u] = 1;
        total += w;
        cnt++;
        for (auto &e : g[u])
            if (!vis[e.first]) pq.push({e.second, e.first});
    }
    return (cnt == n) ? total : -1;
}
```

适合稠密图。n 到 $1e3$ 以上、m 到 n^2 量级时，Kruskal 要排序 m 条边会慢，Prim 反而更快。

易错点

Prim 的优先队列里存的是"边权, 目标点"，弹出时如果该点已经进树就跳过。这个判断不能省。

两种算法选出来的最小生成树总权值相同，但具体选了哪些边可能不同。题目要求输出具体方案时，按题目指定的算法来。

第六章 动态规划

适用情形：题目问"最大 / 最小 / 有多少种方案"，而且答案可以从更小的子问题推出来，子问题之间有重叠。看到"前 i 个""用前 i 种物品""区间"这类描述就该想到 DP。

五步法

第一步，**定义状态**。用一个或几个下标的函数表示"某种局面的最优值或方案数"。例如 $dp[i]$ 表示"前 i

个元素能取得的最大值”， $dp[i][j]$ 表示“前 i 个物品放进容量 j 的背包能取得的最大价值”。

第二步，**写转移方程**。考虑最后一步决策，把所有可能的情况列出来取最大或求和。例如

$dp[i] = \max(dp[i-1], dp[i-1] + a[i])$ 。

第三步，**确定初值**。该填 0 的填 0，该填“无穷大”的填 INF，该填“负无穷”的填 -INF。方案数类问题通常 $dp[0] = 1$ 。

第四步，**确定遍历顺序**。要保证算 $dp[i]$ 时它依赖的状态已经算好了。这和循环的方向、01 背包要从大到小、区间 DP 要先枚举长度都有关。

第五步，**确定答案位置**。是 $dp[n]$ 、 $dp[n][m]$ 还是所有状态里的最大值？有些题答案不是最后一个状态。

复杂度：状态数乘上每个状态转移的次数。

易错点

初值写错是最常见的错误。特别是“恰好装满”和“至多装”两种问法，初值完全不同：恰好装满时

$dp[0] = 0$ 、其他是 -INF；至多装时全部是 0。

6.2 线性 DP

最长上升子序列 LIS

适用情形：在序列里找一个子序列，元素下标递增、值也递增，要求长度最大。

$O(n^2)$ 版本（好写， n 不超过 5000 时用）

```
int dp[MAXN];
int ans = 0;
for (int i = 1; i <= n; i++) {
    dp[i] = 1; // 至少是它自己
    for (int j = 1; j < i; j++)
        if (a[j] < a[i]) dp[i] = max(dp[i], dp[j] + 1);
    ans = max(ans, dp[i]);
}
```

$dp[i]$ 表示以 $a[i]$ 结尾的最长上升子序列长度。答案是所有 dp 里的最大值，不是 $dp[n]$ 。

$O(n \log n)$ 版本（ n 到 $1e5$ 时用）

```
vector<int> tail; // tail[len] = 长度为 len 的上升子序列的最小结尾
for (int i = 1; i <= n; i++) {
    auto it = lower_bound(tail.begin(), tail.end(), a[i]);
    if (it == tail.end()) tail.push_back(a[i]);
    else *it = a[i];
}
int ans = tail.size();
```

原理（文字说明）： tail[len] 记录"所有长度为 len 的上升子序列里，结尾最小的那个值"。结尾越小，将来接上更大的数就越容易。每次读到一个新元素，就用它去替换第一个大于等于它的位置，让结尾变得更小；如果它比所有结尾都大，就说明能接出更长的子序列。

注意： tail 数组里的内容不是真正的最长上升子序列，只是用来计算长度。要输出具体方案得换写法。

严格上升还是非降： lower_bound 得到严格上升（a[i] 可以等于时不替换）， upper_bound 得到非降（允许相等）。

最长公共子序列 LCS

```
// a[1..n], b[1..m]
for (int i = 1; i <= n; i++)
    for (int j = 1; j <= m; j++) {
        if (a[i] == b[j]) dp[i][j] = dp[i-1][j-1] + 1;
        else dp[i][j] = max(dp[i-1][j], dp[i][j-1]);
    }
// 答案 = dp[n][m]
```

dp[i][j] 表示 a 的前 i 个和 b 的前 j 个的最长公共子序列长度。相等时就接上，不相等就丢掉一边的最后一个元素。复杂度 $O(nm)$ 。

最大子段和

```
ll cur = 0, best = -LINF;
for (int i = 1; i <= n; i++) {
    cur = max(cur + a[i], (ll)a[i]); // 要么接着前面，要么从自己重新开始
    best = max(best, cur);
}
// 答案 = best
```

这个写法也叫 Kadane 算法， $O(n)$ 。初值用 $-\text{LINF}$ 而不是 0，因为全是负数时答案应该是最大的那个负数。

6.3 背包

适用情形：有若干个物品，每个物品有重量和价值，背包有容量上限，要求装进去的总价值最大。看到“选或不选”“每种物品若干件”“容量不超过 V ”就是背包。

01 背包

每件物品只有一个，要么选，要么不选。

```
// w[i] 重量, v[i] 价值, V 是容量
for (int i = 1; i <= n; i++)
    for (int j = V; j >= w[i]; j--) // 必须倒着遍历
        dp[j] = max(dp[j], dp[j - w[i]] + v[i]);
// 答案 = dp[V]
```

为什么必须倒着遍历： $dp[j]$ 依赖 $dp[j - w[i]]$ ，也就是下标更小的状态。如果正着遍历， $dp[j - w[i]]$ 可能已经是“这件物品已经放进去过”的状态，等于这件物品被重复使用，就变成了完全背包。倒着走能保证用到的是上一轮的旧值。

记忆口诀：01 背包倒着走，完全背包正着走。

完全背包

每件物品有无限多件。

```
for (int i = 1; i <= n; i++)
    for (int j = w[i]; j <= V; j++) // 正着遍历
        dp[j] = max(dp[j], dp[j - w[i]] + v[i]);
```

和 01 背包只差循环方向。正着遍历时 $dp[j - w[i]]$ 已经包含了“再放一件物品 i ”的可能，等价于物品可以无限取。

多重背包（二进制拆分）

每件物品有 $c[i]$ 件。

朴素做法是把 $c[i]$ 件拆成 $c[i]$ 件独立的物品做 01 背包，复杂度 $O(V \times \text{总件数})$ ，件数大时会超时。二进

制拆分把 $c[i]$ 拆成 1、2、4、8.....这些"包", 每个包里装若干件同种物品, 当成一个 01 物品处理, 复杂度降到 $O(V \times \log c)$ 。

```
for (int i = 1; i <= n; i++) {
    int cnt = c[i], k = 1;
    while (cnt > 0) {
        int use = min(k, cnt);           // 这一包里有 use 件
        ll bw = use * w[i];              // 这一包的重量
        ll bv = use * v[i];              // 这一包的价值
        for (int j = V; j >= bw; j--)
            dp[j] = max(dp[j], dp[j - bw] + bv);
        cnt -= use;
        k <<= 1;
    }
}
```

原理：任何不超过 $c[i]$ 的件数都能表示成若干个 2 的幂的和, 所以按 1、2、4.....拆分后, 任意取法都能拼出来。

分组背包

物品分成若干组, 每组最多选一个。

```
for (int g = 1; g <= G; g++)           // 枚举组
    for (int j = V; j >= 0; j--)        // 容量倒序, 放在组内物品循环的外面
        for (int k = 0; k < (int)items[g].size(); k++) {
            int w = items[g][k].first, v = items[g][k].second;
            if (j >= w) dp[j] = max(dp[j], dp[j - w] + v);
        }
```

关键：容量循环必须在组内物品循环的**外面**。这样才能保证每组只被选一个。如果反过来写, 就变成了每组可以选多个。

方案数

把 max 换成加法, 初值 $dp[0] = 1$:

```

dp[0] = 1;
for (int i = 1; i <= n; i++)
    for (int j = V; j >= w[i]; j--)
        dp[j] = (dp[j] + dp[j - w[i]]) % MOD;

```

如果要求"恰好装满", 初值只给 $dp[0] = 1$, 其他是 0; 如果要求"不超过容量", 答案要把所有 $dp[j]$ 加起来。

易错点

01 背包和完全背包的循环方向写反是最高频的错误。记住"01 倒着, 完全正着"

求方案数时要取模, 而且不能把 max 和加法混在一起。

dp 数组大小要开到 $V+1$, V 到 $1e5$ 时一维数组没问题, 二维数组会 MLE。

多重背包的二进制拆分里, $use = \min(k, cnt)$ 这一句不能省, 否则最后一包会超出剩余件数。

6.4 区间 DP

适用情形: 问题的最优解由某个区间的最优解组合而成, 典型题面是"合并石子""括号匹配""回文串切割"。看到"把区间 $[i, j]$ 分成两半"就要想到区间 DP。

核心思想: $dp[i][j]$ 表示区间 $[i, j]$ 的最优值。转移时枚举一个分割点 k , 把区间分成 $[i, k]$ 和 $[k+1, j]$ 两部分合并。

遍历顺序: 必须先枚举区间长度, 再枚举左端点。因为长度为 len 的区间依赖的是长度更小的区间, 按长度从小到大算能保证依赖的状态已经算好。如果按左端点遍历, 会用到还没算的状态。

石子合并

```

11 dp[505][505];
11 pre[505];                                // 前缀和，用于快速求区间和

for (int i = 1; i <= n; i++) dp[i][i] = 0;   // 长度为 1 的区间，合并代价为 0

for (int len = 2; len <= n; len++) {        // 枚举区间长度
    for (int i = 1; i + len - 1 <= n; i++) { // 枚举左端点
        int j = i + len - 1;                // 右端点
        dp[i][j] = LINF;
        for (int k = i; k < j; k++) {        // 枚举分割点
            11 cost = dp[i][k] + dp[k+1][j] + (pre[j] - pre[i-1]);
            dp[i][j] = min(dp[i][j], cost);
        }
    }
}
// 答案 = dp[1][n]

```

接口说明： $dp[i][j]$ 是从 i 到 j 这一段合并成一堆的最小代价， pre 是前缀和用来 $O(1)$ 求区间和。答案是 $dp[1][n]$ 。

复杂度： 状态数 $O(n^2)$ ，每个状态枚举分割点 $O(n)$ ，总共 $O(n^3)$ 。 n 到 300 时勉强能过， n 到 500 就要考虑优化。

易错点

循环顺序不能写反，必须先长度后左端点。

长度是 1 的区间要初始化成 0，长度为 0 的区间通常不用。

$dp[i][j]$ 求最小值时初值用 $LINF$ ，求最大值时用 $-LINF$ ，不能想当然用 0。

区间 DP 通常是 $O(n^3)$ ，写之前先看数据范围。 n 超过 500 基本就要换算法。

6.5 记忆化搜索

适用情形： 状态转移关系清楚，但遍历顺序不好确定，或者状态只用得到一部分。也常用于 T4 拿小数数据分：先把暴力写成递归，再加记忆化就能过一部分大数据点。

核心思想： 写一个递归函数，参数就是 DP 的状态。进入函数时先查表，如果算过就直接返回；没算过就递归计算，算完存进表里。

复杂度： 和递推 DP 一样，每个状态只算一次。

```

ll memo[1005][1005];
bool vis[1005][1005];
// LINF 直接用 1.1 骨架里定义的全局常量，不要在这里重复定义

ll dfs(int i, int j) {
    if (/* 边界条件 */ return /* 边界值 */;
    if (vis[i][j]) return memo[i][j];          // 算过就直接返回
    vis[i][j] = true;

    ll res = LINF;
    // 枚举所有转移，取最优或求和
    res = min(res, dfs(...) + 代价);

    return memo[i][j] = res;
}

```

接口说明： memo 存答案，vis 标记是否算过。用单独的 vis 数组比用 memo = -1 表示未计算更安全，因为答案本身可能就是 -1。

调用时直接 cout << dfs(1, n); ，不需要额外的外层循环。

第七章 常用处理与易错清单

7.1 高精度大整数

适用情形： 数值超过 long long 的范围（约 9.2×10^{18} 次方），

```

string addBig(string a, string b) {
    string r;
    int i = (int)a.size() - 1, j = (int)b.size() - 1, carry = 0;
    while (i >= 0 || j >= 0 || carry) {
        int s = carry;
        if (i >= 0) s += a[i--] - '0';
        if (j >= 0) s += b[j--] - '0';
        r += char('0' + s % 10);
        carry = s / 10;
    }
    reverse(r.begin(), r.end());
    return r;
}

```

从最低位开始逐位相加，carry 是进位。结果是从低位往高位追加的，所以最后要 reverse。循环条件里的 || carry 保证最后一次进位也能处理到。

减法

```
// 要求 a >= b, 且都是非负大整数
string subBig(string a, string b) {
    string r;
    int i = (int)a.size() - 1, j = (int)b.size() - 1, borrow = 0;
    while (i >= 0) {
        int s = (a[i] - '0') - borrow;
        if (j >= 0) s -= (b[j--] - '0');
        if (s < 0) { s += 10; borrow = 1; }
        else borrow = 0;
        r += char('0' + s);
        i--;
    }
    reverse(r.begin(), r.end());
    return trim(r);
}
```

7.2 暴力枚举模板库

适用情形：n 很小（不超过 8 到 20），或者需要在小数据点上拿分。看到"输出所有方案""求第 k 个""n 不超过 20"就直接上暴力。

DFS 全排列

```

int n, perm[15];
bool used[15];
void dfs(int d) {
    if (d > n) {
        // 得到一个完整排列 perm[1..n], 在这里处理
        return;
    }
    for (int i = 1; i <= n; i++) {
        if (used[i]) continue;
        used[i] = true;
        perm[d] = i;
        dfs(d + 1);
        used[i] = false;           // 回溯: 恢复现场, 这一句不能省
    }
}

// 调用: dfs(1);

```

接口说明: d 是当前要填的位置

DFS 组合 (从 n 个里选 k 个)

```

int n, k;
vector<int> chosen;
void dfs(int start, int depth) {
    if (depth == k) {
        // 处理 chosen
        return;
    }
    for (int i = start; i <= n; i++) {
        chosen.push_back(i);
        dfs(i + 1, depth + 1);
        chosen.pop_back();         // 回溯
    }
}

// 调用: dfs(1, 0);

```

网格 DFS (连通块、岛屿数量)

```

int n, m;
int dx[4] = {-1, 1, 0, 0};
int dy[4] = {0, 0, -1, 1};
char g[1005][1005];
bool vis[1005][1005];

void dfs(int x, int y) {
    vis[x][y] = true;
    for (int d = 0; d < 4; d++) {
        int nx = x + dx[d], ny = y + dy[d];
        if (nx < 0 || nx >= n || ny < 0 || ny >= m) continue;
        if (vis[nx][ny] || g[nx][ny] == '#') continue;
        dfs(nx, ny);
    }
}

```

可行性剪枝：如果当前状态已经不可能到达合法解，直接返回。例如背包搜索里"当前重量已经超过容量"。

最优性剪枝：如果当前代价已经不比已知最优解好，直接返回。配合一个全局的 best 变量。

顺序剪枝：先搜更可能出解的分支，让最优解尽快被找到，后面的分支就能被剪掉更多。

记忆化：同一个状态不重复计算，等价于给搜索加了缓存。代码见 6.5。